Deakin University

Concurrent and Distributed Programming

OnTrack Submission

Comprehensive Interrupt-Based Sense-Think-Act System

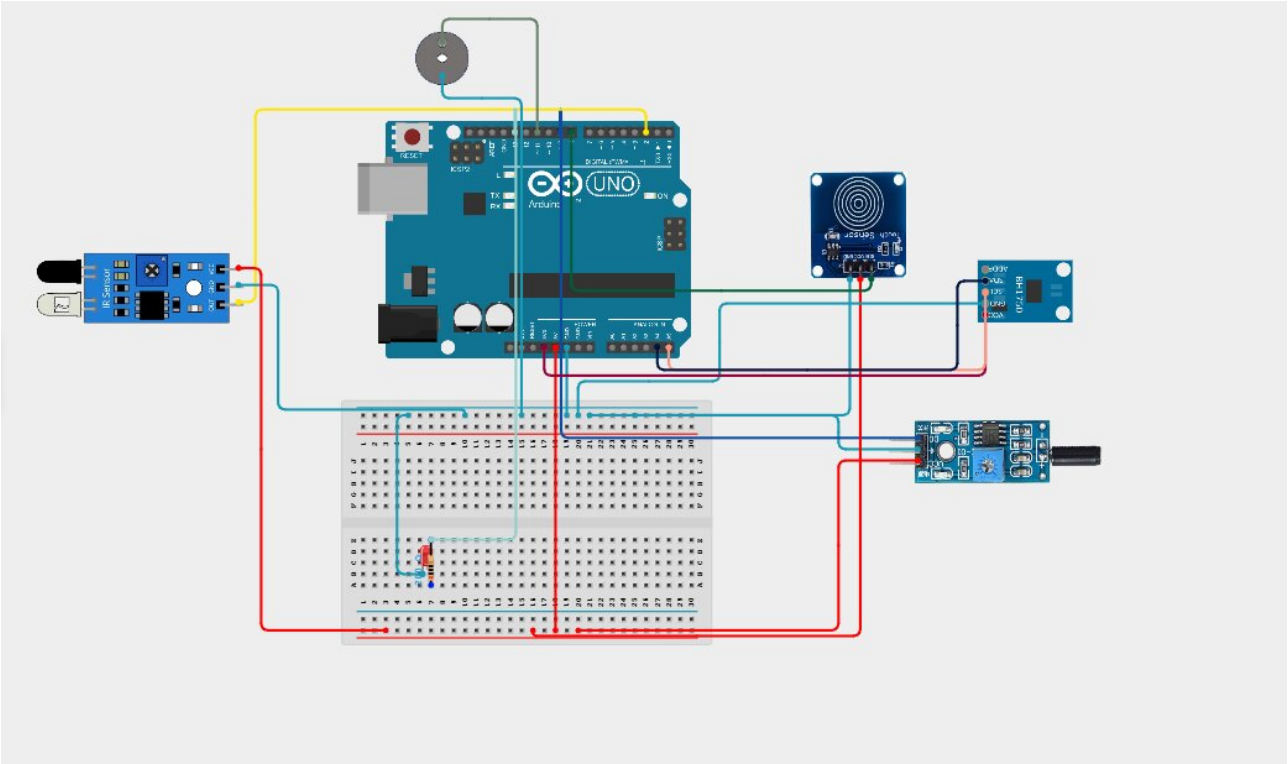
Submitted By:
Manit Khera
s224520987
2026/08/03 15:48

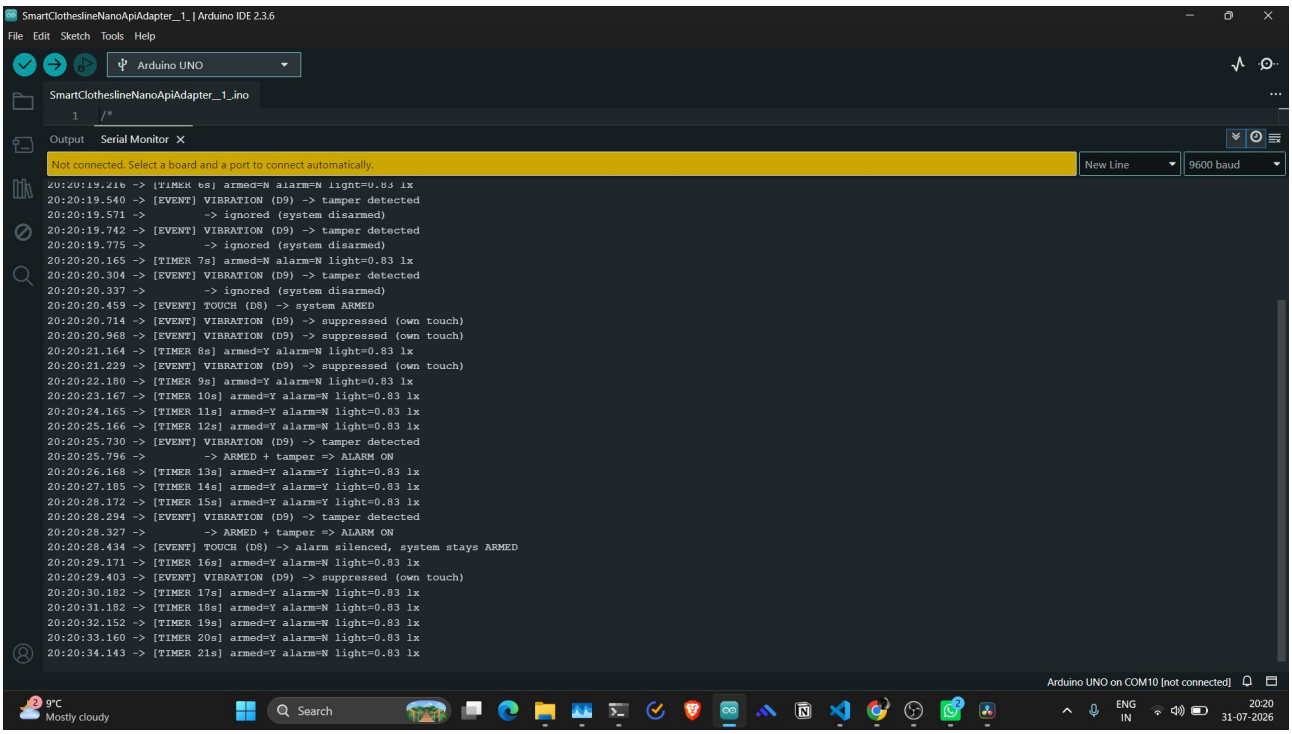
Tutor:
Clement Asogwa

Task Outcomes		Supports
TLO1	Demonstrate engagement with the following unit learning outcomes (legacy)	ULO2 Design and Implement

August 3, 2026




UNO
Arduino
IR Sensor
Sensor
BH1750


SmartClotheslineNanoApiAdapter__1_ | Arduino IDE 2.3.6
File Edit Sketch Tools Help
Arduino UNO
SmartClotheslineNanoApiAdapter__1_.ino
Output Serial Monitor
Not connected. Select a board and a port to connect automatically.
New Line
9600 baud
20:20:19.540 -> [EVENT] VIBRATION (D9) -> tamper detected
20:20:19.571 -> -> ignored (system disarmed)
20:20:19.742 -> [EVENT] VIBRATION (D9) -> tamper detected
20:20:19.775 -> -> ignored (system disarmed)
20:20:20.165 -> [TIMER 7s] armed=N alarm=N light=0.83 lx
20:20:20.304 -> [EVENT] VIBRATION (D9) -> tamper detected
20:20:20.337 -> -> ignored (system disarmed)
20:20:20.459 -> [EVENT] TOUCH (D8) -> system ARMED
20:20:20.714 -> [EVENT] VIBRATION (D9) -> suppressed (own touch)
20:20:20.968 -> [EVENT] VIBRATION (D9) -> suppressed (own touch)
20:20:21.164 -> [TIMER 8s] armed=Y alarm=N light=0.83 lx
20:20:21.229 -> [EVENT] VIBRATION (D9) -> suppressed (own touch)
20:20:22.180 -> [TIMER 9s] armed=Y alarm=N light=0.83 lx
20:20:23.167 -> [TIMER 10s] armed=Y alarm=N light=0.83 lx
20:20:24.165 -> [TIMER 11s] armed=Y alarm=N light=0.83 lx
20:20:25.166 -> [TIMER 12s] armed=Y alarm=N light=0.83 lx
20:20:25.730 -> [EVENT] VIBRATION (D9) -> tamper detected
20:20:25.796 -> -> ARMED + tamper => ALARM ON
20:20:26.168 -> [TIMER 13s] armed=Y alarm=Y light=0.83 lx
20:20:27.185 -> [TIMER 14s] armed=Y alarm=Y light=0.83 lx
20:20:28.172 -> [TIMER 15s] armed=Y alarm=Y light=0.83 lx
20:20:28.294 -> [EVENT] VIBRATION (D9) -> tamper detected
20:20:28.327 -> -> ARMED + tamper => ALARM ON
20:20:28.434 -> [EVENT] TOUCH (D8) -> alarm silenced, system stays ARMED
20:20:29.171 -> [TIMER 16s] armed=Y alarm=N light=0.83 lx
20:20:29.403 -> [EVENT] VIBRATION (D9) -> suppressed (own touch)
20:20:30.182 -> [TIMER 17s] armed=Y alarm=N light=0.83 lx
20:20:31.182 -> [TIMER 18s] armed=Y alarm=N light=0.83 lx
20:20:32.152 -> [TIMER 19s] armed=Y alarm=N light=0.83 lx
20:20:33.160 -> [TIMER 20s] armed=Y alarm=N light=0.83 lx
20:20:34.143 -> [TIMER 21s] armed=Y alarm=N light=0.83 lx
Arduino UNO on COM10 [not connected]
9°C
Mostly cloudy
Search
ENG
IN
20:20
31-07-2026

```cpp

#include <Wire.h>

// ------------------------ Pin assignments --------------------------------
const uint8_t PIN_IR         = 2;
const uint8_t PIN_TOUCH      = 8;
const uint8_t PIN_VIBRATION  = 9;
const uint8_t PIN_ALARM_LED  = 13;
const uint8_t PIN_STATUS_LED = 12;
const uint8_t PIN_BUZZER     = 11;

const bool BUZZER_IS_ACTIVE = true;
const uint16_t BUZZER_FREQ  = 2000;

const uint8_t BH1750_ADDR    = 0x23;

volatile bool     irFlag        = false;
volatile bool     touchFlag     = false;
volatile bool     vibrationFlag = false;
volatile uint32_t tickCount     = 0;
volatile bool     timerFlag     = false;

volatile uint8_t  prevPINB      = 0;

bool  systemArmed = false;
bool  alarmActive = false;
bool  buzzerOn    = false;
float lightLux    = 0.0;

volatile uint32_t lastTouchMs     = 0;
volatile uint32_t lastVibrationMs = 0;
const uint32_t DEBOUNCE_MS = 200;

uint32_t touchHandledMs   = 0;
const uint32_t TAMPER_SUPPRESS_MS = 1500;

//----------------------------------INterrupt_functios-----------------------------
----------------

//Interupt function for when IR sensor pciks up somehting
void isrObstacle()
{
  irFlag = true;
}

//Interuppt function for when a touch or a vibration occurs(both sensors share this
 ISR function)
ISR(PCINT0_vect) //PCINT0_vect means port b, PCINT1_vect means port C and
 PCINT2_vect means port d. The physical port, not register
{
  uint8_t  pinb    = PINB;      // Copy the current HIGH/LOW states of all Port B
   pins

// Compare the current pin states with the previous states.
```

```
51  // Bits become 1 wherever a pin changed
52    uint8_t  changed = pinb ^ prevPINB;  //XOR
53
54  // Save the current pin states so they can be used as the "previous" states
55  // during the next pin-change interrupt.
56    prevPINB         = pinb;
57    uint32_t now     = millis();
58
59
60  //If D8 changed to HIGH:
61      //touchFlag = true
62    if ((changed & 0b00000001) && (pinb & 0b00000001))
63    {
64      if (now - lastTouchMs > DEBOUNCE_MS) {
65        touchFlag   = true;
66        lastTouchMs = now;
67      }
68    }
69
70    //If D9 changed to LOW:
71      //vibrationFlag = true
72    if ((changed & 0b00000010) && !(pinb & 0b00000010)) {
73      if (now - lastVibrationMs > DEBOUNCE_MS) {
74        vibrationFlag   = true;
75        lastVibrationMs = now;
76      }
77    }
78  }
79
80
81  //Interupt function for when 1 second passes
82  ISR(TIMER1_COMPA_vect)
83  {
84    tickCount++;
85    timerFlag = true;
86  }
87
88
89  //-------------------------------------------------------------------------Setup------
    ↪  ------------------------------------------
90  void setup()
91  {
92    Serial.begin(9600);
93    while (!Serial) { ; }
94
95
96    pinMode(PIN_IR,         INPUT);
97    pinMode(PIN_TOUCH,      INPUT);
98    pinMode(PIN_VIBRATION, INPUT);
99    pinMode(PIN_ALARM_LED,  OUTPUT);
100   pinMode(PIN_STATUS_LED, OUTPUT);
101   pinMode(PIN_BUZZER,     OUTPUT);
102
103   Wire.begin();
```

```
104   bh1750Begin();
105
106
107   attachInterrupt(digitalPinToInterrupt(PIN_IR), isrObstacle, FALLING);
108
109
110   configurePinChangeInterrupts();
111
112
113   configureTimer1_1Hz();
114
115   Serial.println(F("=== SIT315 M1 Sense-Think-Act (4/4) online ==="));
116   Serial.println(F("D2=FC-51 IR (INT0)  D8=TOUCH (PCI)  D9=VIBRATION (PCI)
      ↪  I2C=BH1750"));
117   Serial.println(F("System starts DISARMED. Touch the pad to arm."));
118   Serial.println(F("While the alarm is ringing, touch silences it (stays
      ↪  armed).\n"));
119 }
120
121
122
123 /// ------------------------------------------------------------------------CONfig--
      ↪  ----------------------------------------
124
125
126 void configurePinChangeInterrupts()
127 {
128   prevPINB = PINB;
129
130   PCICR |= 0b00000001;// enable group 0 for pin change interrupt
131   PCMSK0 |= 0b00000011; //Select which of the pins in the group can actually trigger
      ↪  interuppt.
132 }
133
134 //T
135 void configureTimer1_1Hz()
136 {
137   noInterrupts();
138
139   TCCR1A = 0b00000000;  // clear Timer1 control register A
140   TCCR1B = 0b00000000;  // clear Timer1 control register B
141   TCNT1  = 0;           // start counting from zero
142
143   OCR1A = 15624;        // compare value for 1-second interval
144
145   TCCR1B |= 0b00001000; // enable CTC mode: WGM12 = 1
146   TCCR1B |= 0b00000101; // prescaler 1024: CS12 = 1, CS10 = 1
147
148   TIMSK1 |= 0b00000010; // enable Timer1 Compare-A interrupt
149
150   interrupts();
151 }
152
153
```

```
154
155 //-----------------------------------------------loop-----------------------------------
    ↪  --------------------------------
156
157 void loop() {
158   handleEvents(); // handles flag changes
159   handlePeriodic();   //handles periodic events
160   updateActuators();
161 }
162
163
164
165 //-------------------------------------------------------fucntion inside
    ↪  loop-----------------------------------------------
166
167 void handleEvents() {
168   if (touchFlag==true)
169   {
170
171     //Set the flag to false again
172     touchFlag = false;
173
174
175     touchHandledMs = millis();
176     if (alarmActive==true)
177     {
178       alarmActive = false; //Set the alarmActive flag to false or OFF
179       Serial.println(F("[EVENT] TOUCH (D8) -> alarm silenced, system stays ARMED"));
180     } else
181     {
182       systemArmed = !systemArmed; //toggle systemArmed flag
183       Serial.print(F("[EVENT] TOUCH (D8) -> system "));
184       Serial.println(systemArmed ? F("ARMED") : F("DISARMED"));
185     }
186   }
187
188
189   if (vibrationFlag==true)
190   {
191
192     //Set the flag to false again
193     vibrationFlag = false;
194
195
196     if (millis() - touchHandledMs < TAMPER_SUPPRESS_MS) //if the vibration is 1.5s
    ↪  of a touch, then move on.
197     {
198       Serial.println(F("[EVENT] VIBRATION (D9) -> suppressed (own touch)"));
199     }
200     else //else run the actual if-else condition
201     {
202       Serial.println(F("[EVENT] VIBRATION (D9) -> tamper detected"));
203       if (systemArmed==true) //if system is armed
204       {
```

```
205         alarmActive = true; //then make alarm active
206         Serial.println(F("        -> ARMED + tamper => ALARM ON"));
207       } else
208       {
209         Serial.println(F("        -> ignored (system disarmed)"));
210       }
211     }
212   }
213
214   if (irFlag==true)
215   {
216
217     //Set the flag to false again
218     irFlag = false;
219
220
221     Serial.println(F("[EVENT] OBSTACLE on D2 (INT0) -> intrusion"));
222     if (systemArmed)
223     {
224       alarmActive = true;
225       Serial.println(F("        -> ARMED + intrusion => ALARM ON"));
226     } else {
227       Serial.println(F("        -> ignored (system disarmed)"));
228     }
229   }
230 }
231
232
233
234 void handlePeriodic()
235 {
236   if (!timerFlag) return;            // nothing to do until a second elapses
237   timerFlag = false;
238
239   // Safely copy the multi-byte shared counter (disable IRQs for the read).
240   uint32_t ticks;
241   noInterrupts();
242   ticks = tickCount; // if isr runs then tickcount may get stuck, so uses stable
      ↪  local copy
243   interrupts();
244
245   lightLux = bh1750ReadLux();
246
247   digitalWrite(PIN_STATUS_LED, !digitalRead(PIN_STATUS_LED));
248
249   buzzerOn = !buzzerOn;
250
251   Serial.print(F("[TIMER ")); Serial.print(ticks);
252   Serial.print(F("s] armed=")); Serial.print(systemArmed ? F("Y") : F("N"));
253   Serial.print(F(" alarm="));   Serial.print(alarmActive ? F("Y") : F("N"));
254   Serial.print(F(" light="));   Serial.print(lightLux); //prints the lux
255   Serial.println(F(" lx"));
256 }
257
```

```
// ------------------------ ACT ------------------------------------------
void updateActuators()
{
  digitalWrite(PIN_ALARM_LED, alarmActive ? HIGH : LOW);

  // Buzzer beeps in time with the 1 Hz tick while the alarm is active, and is
  // silent otherwise. Pulsing carries much better than a continuous tone.
  bool shouldSound = alarmActive && buzzerOn;

  if (BUZZER_IS_ACTIVE) {
    digitalWrite(PIN_BUZZER, shouldSound ? HIGH : LOW);
  } else {
    // tone() drives Timer2 - deliberately NOT Timer1, so it cannot disturb the
    // 1 Hz periodic interrupt this program depends on.
    if (shouldSound) tone(PIN_BUZZER, BUZZER_FREQ);
    else             noTone(PIN_BUZZER);
  }
}


// ------------------------ helper BH1750 fucntions ----------------------------


void bh1750Begin() {
  Wire.beginTransmission(BH1750_ADDR);
  Wire.write(0x01);            // power on
  Wire.endTransmission();
  Wire.beginTransmission(BH1750_ADDR);
  Wire.write(0x10);            // continuous high-resolution mode
  Wire.endTransmission();
  delay(200);                  // settle time - safe here, we are still in setup()
}

float bh1750ReadLux() {
  if (Wire.requestFrom(BH1750_ADDR, (uint8_t)2) != 2) return -1.0;
  uint16_t raw = ((uint16_t)Wire.read() << 8) | Wire.read();
  return raw / 1.2;            // datasheet conversion
}
```

Deakin University
SIT315 - Concurrent and Distributed Programming
Trimester 2, 2026

Module 1: Real-Time and Embedded Systems
Task M1.T1P - Comprehensive Interrupt-Based Sense-Think-Act System
Student Name: Manit Khera
Student ID: s22452098
Submission Date: 31 July 2026

Reflection

I built a small home security panel on an Arduino Uno. An IR sensor on D2 spots anything passing in front of it, a touch pad on D8 arms and disarms it, a vibration switch on D9 feels someone knocking the desk, and a light sensor is read once a second. When something triggers while armed, the onboard LED on D13 and a buzzer go off, and a second LED blinks every second to show the board is alive. Being armed and being triggered are separate things, so the same wave of my hand is ignored when the system is off but alarms when it is on. All three kinds of interrupt are in here. The IR sensor uses attachInterrupt on D2, and because it idles HIGH and drops LOW on detection, I had to trigger on FALLING. The touch pad and the vibration switch are both on Port B, so they share one handler. Timer1 fires once a second, which I got by slowing the 16 MHz clock down by 1024 and counting to 15624. My rule was that a handler does almost nothing: it flips a flag and gets out, and the real work happens in loop(), because waiting or printing inside an interrupt is how you miss the next one. My light sensor gave no error at all, just failed reads. A bus scanner came back empty, and since a wrong address would still have shown a device, that had to mean wiring: two wires were swapped. Pressing the touch pad shook the desk enough for the vibration sensor to feel it, so silencing the alarm instantly set it off again as tamper. Two sensors I thought were separate were coupled through the table. I fixed it with the sensitivity pot and a short window after a touch where a shake is logged but ignored. The worst bug was the quietest. Because those two sensors share one handler, I was checking whether the pins were HIGH or LOW rather than whether they had changed, so when the vibration sensor fired the code saw my finger still on the touch pad and counted it as a new press. The system was arming itself on presses I never made. Debouncing could not help, since a real press lasts about half a second against my 200 ms window. The fix was to remember what the pins looked like last time and compare, so each sensor reacts only when its own pin moved. I also lost the line that enables the timer interrupt once, and learned that configuring a timer is not the same as switching it on. Two choices I would defend: the touch pad and vibration switch read opposite ways on purpose, so the shared handler has to work out which one fired, and the buzzer runs off a

different timer so it cannot disturb the one second tick. The proof it works is the log, where the timer count never skips a number even when several events land in one second.

Github link: https://github.com/uiqvb/Interrupt-Driven-Intrusion-and-Tamper-Alarm

Video link: https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=71cb82cb-7f2c-42c7-9cca-b49800c22007